



COMSATS University Islamabad, Lahore Campus

Department of Computer Sciences

Terminal Lab – Semester Spring 2026

Course Title	Software Testing			Course Code	CSE482	Credit Hours:	3
Course Instructor	Ms. Yella Mehroze			Program Name	BSE		
Semester	8 th	Batch:	FA22-BSE	Section	A	Type	Theory
Total Marks	50	Obtained Marks:		Date	May 09, 2026		
Student's Name	UMAIR ALI			Reg. No	FA22-BSE-137		

Question1:

[Marks 20]

[CLO <5>]: Create test plan document for a medium size software system in a team environment.
Bloom Taxonomy Level<Create>

Part B - Choose the Cypress framework to perform End-to-End (E2E) Testing.

Task 1 — UI Test Suite (Functional Testing)

Goal: Write Cypress test cases for a real web application and practise core Cypress commands.

Choose a Website to Test

You may test any publicly accessible website of your choice — including your own project or a university portal. The following are suggested practice sites if you need a starting point:

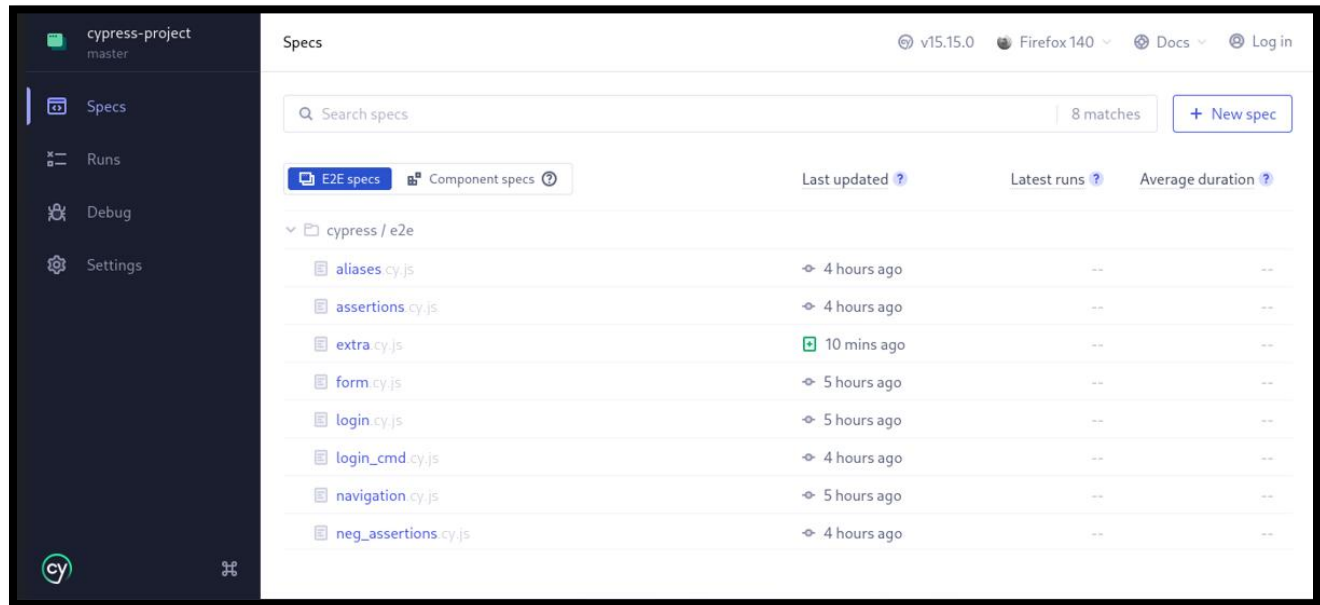
- <https://www.saucedemo.com> — Simple login & shopping app (Best for beginners)
- <https://the-internet.herokuapp.com> — QA playground with forms, alerts, and dropdowns
- <https://demo.opencart.com> — Full e-commerce storefront
- Any other website you regularly use or find interesting

Required Test Cases [Students are free to do any two from both , like 2 login tests and 2 navigation tests]

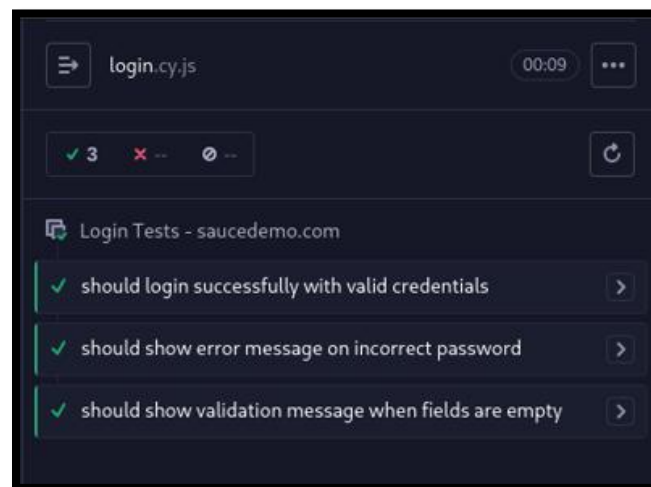
Write the following 6 test cases. Each must use `cy.visit()`, `cy.get()`, and `cy.should()` at minimum.

- **Login Test 1:** Enter valid username and password. Assert that the user lands on the home or dashboard page after login.
- **Login Test 2:** Enter an incorrect password. Assert that an error message appears on the screen.
- **Login Test 3:** Leave both fields empty and click login. Assert that a validation message is shown.
- **Navigation Test 1:** Click on any menu link. Assert the correct page opens and the URL or heading matches.
- **Navigation Test 2:** Visit 2 different pages in sequence. Assert each page has the correct title or visible heading.
- **Form Test 1:** Fill out any form on the site (search, contact, or registration). Submit it and assert the result — a success message, results page, or confirmation.

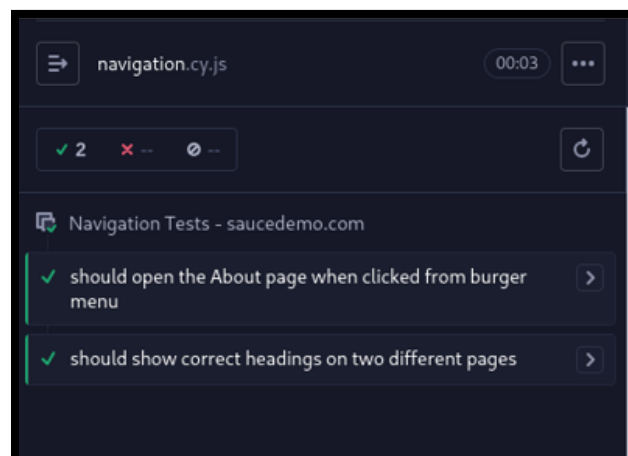
CYPRESS UI



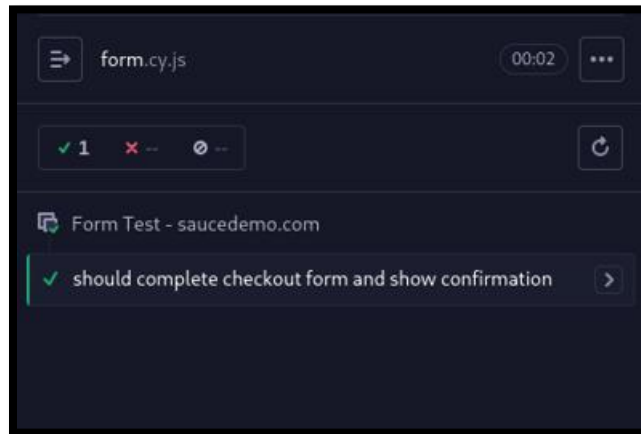
LOGIN TEST



NAVIGATION TEST



FORM TEST

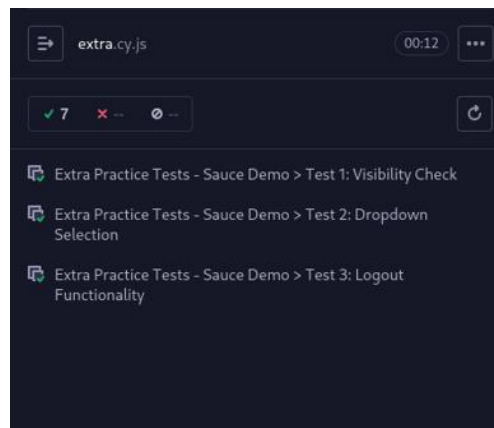


Extra Practice [Not mandatory – Bonus Tasks]

The following are not required but are strongly encouraged to build your confidence with Cypress:

- Write a test that checks whether a specific button, image, or text is visible on the page.
- Write a test that clicks a dropdown and selects an option, then asserts the page updates correctly.
- Write a test that logs out and asserts the login page is shown again.
- Use `cy.screenshot()` inside a test to capture the page at any point during execution.
- Try writing tests for a website you use daily — a university portal, news site, or online store.
- Explore `cy.contains()` to find elements by their text content instead of a CSS selector.

EXTRA TASKS



GitHub Repo Link: <https://github.com/umair-fa22/cypress-project>

What to Submit

1. A GitHub repository containing your Cypress project
2. A README file explaining how to run the tests
3. At least one screenshot of your tests passing in the terminal or browser

Organise your files inside the `cypress/e2e/` folder. Name each file clearly – for example `login.cy.js` or `navigation.cy.js`.

Task 2 — Assertions, Aliases & Custom Commands

Goal: Go deeper into Cypress by practising different types of assertions, using aliases, and writing reusable custom commands.

Choose a website to Test

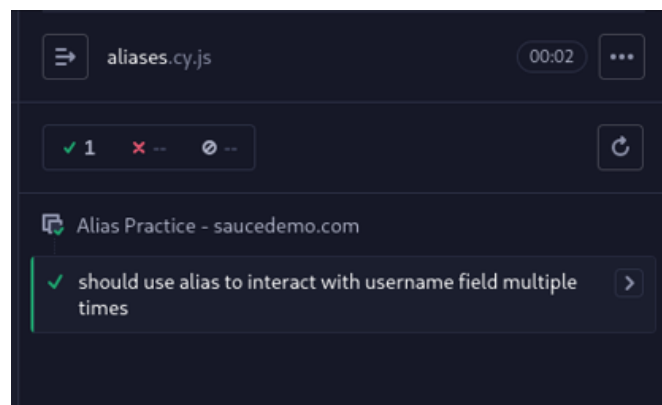
You may use the same site from Task 1 or pick a different one. Any publicly accessible website works.

Required Exercises

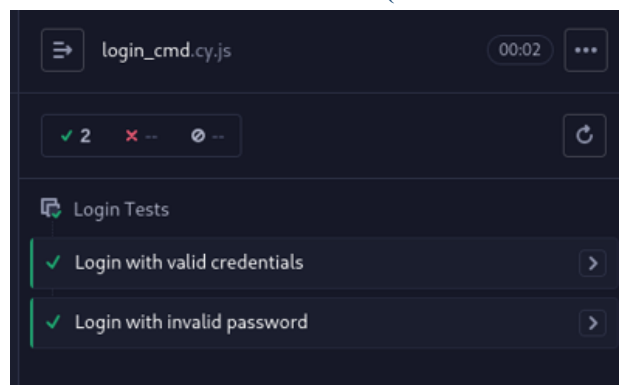
Complete the following exercises. Each one builds a specific Cypress skill.

- **Assertion Practice:** Write 3 tests using different assertion types — `cy.should('be.visible')`, `cy.should('have.text', '...')`, and `cy.should('have.attr', '...')`. Each must target a different element on the page.
- **Negative Assertion:** Write a test that asserts something does NOT exist or is NOT visible on the page using `cy.should('not.exist')` or `cy.should('not.be.visible')`.
- **Alias Practice:** Use `cy.get(...).as('myAlias')` to save an element as an alias. Then use `cy.get('@myAlias')` later in the same test to interact with it again. Write at least 1 test demonstrating this.
- **Custom Command:** Create at least 1 custom command in `cypress/support/commands.js` — for example a `login()` command that fills the username, password, and clicks submit. Use it in a test instead of repeating those steps manually.
- **beforeEach Hook:** Use a `beforeEach()` block to run `cy.visit()` before every test in one of your spec files so you do not repeat it in each test individually.

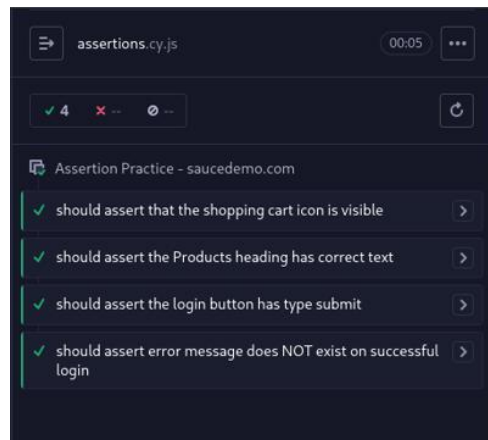
ALLIAS



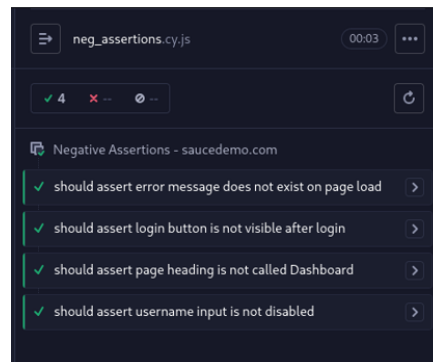
LOGIN CUSTOM COMMAND (USING CY.LOGIN())



ASSERTION



NEGATIVE ASSERTION



GitHub Repo Link: <https://github.com/umair-fa22/cypress-project>

One of the most challenging parts of this project was setting up custom commands in Cypress. Initially, `cy.login()` was throwing a "not a function" error even though the command was correctly defined in `cypress/support/commands.js`. After investigating, I discovered that the `cypress/support/e2e.js` file was not importing the commands file, which meant Cypress never loaded the custom commands at runtime. I fixed this by adding `import './commands'` at the top of `e2e.js`, which immediately resolved the error. This taught me that in Cypress, defining a custom command and registering it are two separate steps, and both are required for the command to work in tests.

What to Submit

1. Updated GitHub repository with Task 2 spec files and a custom commands file
2. A README update explaining what Task 2 covers and how to run those tests
3. A short paragraph (4–6 sentences) describing one thing you found difficult and how you solved it

Keep Task 1 and Task 2 test files separate. A clear folder structure makes your work easier to review and shows good professional habits.